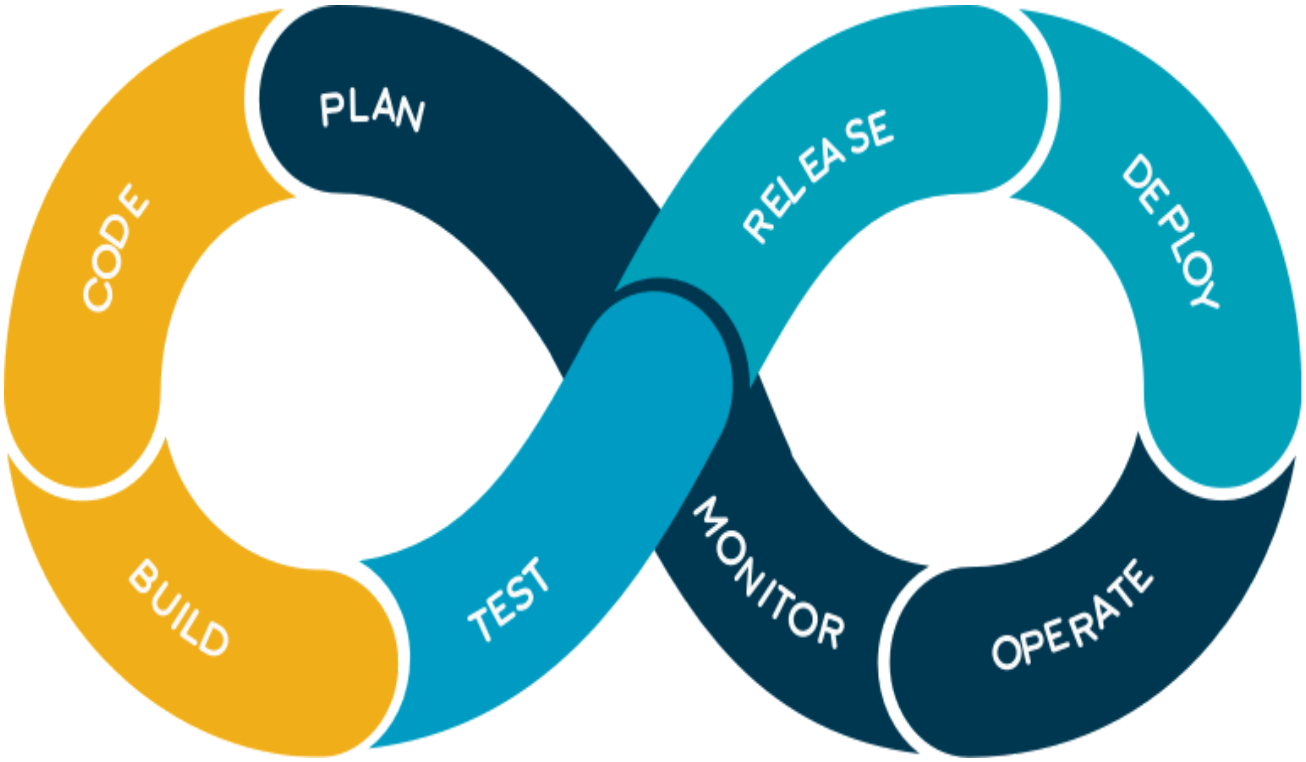
Unit IV DevOps:



What is DevOps?

DevOps is the combination of **cultural philosophies**, practices, and tools that increase an organization's ability to deliver applications and services at **high velocity**: evolving and improving products at a faster pace than organizations using traditional software development and infrastructure management processes. This speed enables organizations to better serve their customers and compete more effectively in the market.

AWS provides a set of flexible services designed to enable companies to more rapidly and reliably build and deliver products using AWS and DevOps practices. These services simplify provisioning and managing infrastructure, deploying application

code, automating software release processes, and monitoring your application and infrastructure performance.

DevOps, a new organizational and cultural way of organizing development and IT operations work, and its **sister technologies**, **continuous integration and continuous deployment (CI/CD),** have transformed the way we create software.

"DevOps" is from a linguistic point of view a portmanteau– a new word created of merging two already existing ones: "development" and "operations". This also reflects a general idea behind it. Software developers and the rest of teams are directly involved in a process leading to creation something new out of existing resources.

How Do DevOps Methodologies Work?

DevOps methodology of software development is all about automating the software development process to speed up delivery and make it more efficient. DevOps methodologies involve **communication, collaboration and integration** between **software developers and IT operations professionals**. The goal is to make the software development process more efficient by automating repetitive tasks, eliminating errors and reducing the time it takes to deliver new features to users.

DevOps relies on several tools and technologies to achieve these objectives, including continuous integration (CI), continuous delivery (CD) and Infrastructure-as-Code (IaC). By automating the **build, test and deploy process,** DevOps teams can release new software updates frequently and with minimal disruption.

***□ Cultural philosophy:□**

Cultural philosophy is defined as the study of the symbolic and behavioural aspects of culture, focusing on community-specific ideas about truth, goodness, beauty, and efficiency that are socially inherited and play a role in shaping different ways of life.

DevOps is a methodology of software development that has a significant impact on the entire development process, particularly in relation to speed and quality. Here's how it impacts software development in detail.

Continuous Development

This is the first step in the DevOps process. It involves writing code and then committing it to a central repository. This code is then automatically built and tested. If there are no errors, it is then deployed to a staging environment where it can be further tested before being pushed to production.

***What is a Central Repository?**

A central repository is a singular storage location for all data within an organization. It uses a central repository model, where all assets such as code, documents, and other files, are kept in one central place.

***Staging Environment**

The Staging environment is where the code is deployed after it's been written in the development environment but before it reaches the production stage.

Configuration Management

Configuration management is all about keeping track of changes made to the code base. This includes things like tracking who made what changes and when they were made. It also includes keeping track of the different versions of the codebase so that if something goes wrong, it can be rolled back to a previous version.

Continuous Integration

Continuous integration is the process of automating the build and testing of code every time a change is made. This ensures that errors are caught early and that the code base is always in a deployable state.

Continuous Testing

Continuous testing is the process of automatically running tests against the code base regularly. This helps to catch errors early and prevent them from being deployed to production.

Continuous Deployment

Continuous deployment is the process of automating the deployment of code to production. This means that changes can be made and deployed to production quickly and easily without having to go through a lengthy approval process.

Continuous Operations

Continuous operations are the process of keeping the system up and running 24/7. This includes things like monitoring for errors and ensuring that the system can recover from failures quickly.

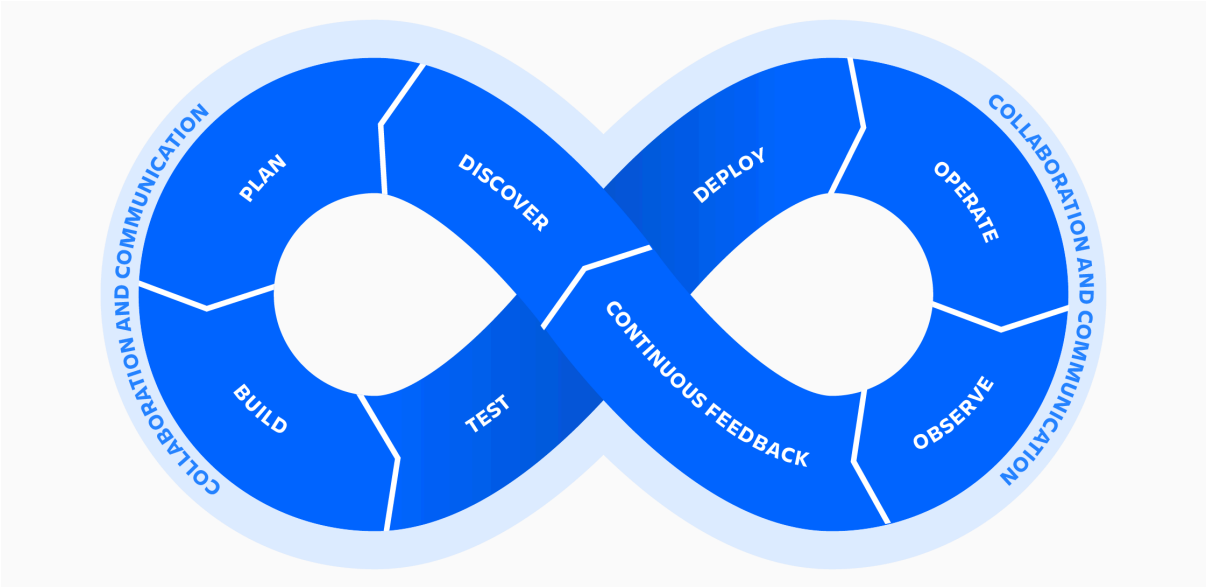
Continuous Monitoring

Continuous monitoring is the process of monitoring the system for performance issues and errors. This helps identify problems early so they can be fixed before they cause any downtime.

DevOps Principles:

To **realize** the full potential of DevOps, teams should follow key DevOps principles

DevOps is more than just development and operations teams working together. It's more than tools and practices. DevOps is a mindset, a cultural shift, where teams adopt new ways of working.



A DevOps culture means developers get closer to the user by gaining a better understanding of user requirements and needs. Operations teams get involved in the development process and add maintenance requirements and customer needs. It means adhering to the following key principles that help DevOps teams deliver applications and services at a faster pace and higher quality than organizations using the traditional software development model.

Collaboration

The key premise behind DevOps is collaboration. Development and operations teams coalesce into a functional team that communicates, shares feedback, and collaborates throughout the entire development and deployment cycle. Often, this means development and operations teams merge into a single team that works across the entire application lifecycle.

The members of a DevOps team are responsible for ensuring quality deliverables across each facet of the product. This leads to more ‘full stack’ development, where teams own the complete backend-to-frontend responsibilities of a feature or product. Teams will own a feature or project throughout the complete lifecycle from idea to delivery. This enhanced level of investment and attachment from the team leads to higher quality output.

Automation

An essential practice of DevOps is to automate as much of the software development lifecycle as possible. This gives developers more time to write code and develop new features. Automation is a key element of a CI/CD pipeline and helps to reduce human errors and increase team productivity. With automated processes, teams achieve continuous improvement with short iteration times, which allows them to quickly respond to customer feedback.

Continuous Improvement

Continuous improvement was established as a staple of agile practices, as well as lean manufacturing and Improvement Kata. It's the practice of focusing on experimentation, minimizing waste, and optimizing for speed, cost, and ease of delivery. Continuous improvement is also tied to continuous delivery, allowing DevOps teams to continuously push updates that improve the efficiency of software systems. The constant pipeline of new releases means teams consistently push code changes that eliminate waste, improve development efficiency, and bring more customer value.

Customer-centric action

DevOps teams use short feedback loops with customers and end users to develop products and services centered around user needs. DevOps practices enable rapid collection and response to user feedback through use of real-time live monitoring and rapid deployment. Teams get immediate visibility into how live users interact with a software system and use that insight to develop further improvements.

Create with the end in mind

This principle involves understanding the needs of customers and creating products or services that solve real problems. Teams shouldn't 'build in a bubble', or create software based on assumptions about how consumers will use

the software. Rather, DevOps teams should have a holistic understanding of the product, from creation to implementation.

DevOps Strategies:

DevOps strategy is based on six fundamental factors, which are:

- **Speed**
  Through our DevOps model, we ensure a faster innovation and speedy execution is key to customer satisfaction and to stay ahead in the competition.

- **Rapid Delivery**
  Our DevOps implementation involves very frequent delivery cycles (which could be small updates) and a minimum recovery time (in case of any failure) through Continuous Integration practice, thus allowing further and faster innovation.

- **Reliability**
  Through DevOps continuous integration and delivery practices, we ensure functional, safe and quality output, resulting in a positive end-user experience. Overall, our DevOps program ensures the productivity of developers and the reliability of operations.

- **Scale**
  Our Infrastructure as a code practice helps in the efficient management of all stages of the software product lifecycle (development, testing and production).

- **Improved Collaboration**
  DevOps methodology is built on a cultural philosophy that focuses on development and operations working in a collaborative environment, which reduces multiple iterations, inefficiency and time complexity.

- **Security**
  Our DevOps model achieves this factor through configuration management techniques and automated compliance policies. Besides, we also work on the concept of DevSecOps, which is a new integration to address the security challenges in DevOps implementation.

Automation:

DevOps	Automation
DevOps is an approach to culture, automation, and platform design intended to deliver increased business value and responsiveness through rapid, high-quality service delivery. DevOps practices bring development and operations team members together into a single DevOps team. This moves ideas and projects from development to production faster and more efficiently. DevOps involves more frequent changes to code and more dynamic infrastructure use when compared to traditional, manual management strategies.	Automation is the use of technology to perform tasks with reduced human assistance. Automation helps you accelerate processes and scale environments, as well as build continuous integration, continuous delivery, and continuous deployment (CI/CD) workflows. There are many kinds of automation, including IT automation, business automation, robotic process automation, industrial automation, artificial intelligence, machine learning, and deep learning.

IT automation

A system of instructions that carry out a repeated set of processes to replace manual work done to IT systems, such as **automating provisioning** using a standard operating environment (SOE).

Business automation

The alignment of **business process management** (BPM), **business process automation** (BPA), **business rules management** (BRM), and **business optimization** with modern application development to respond to market changes.

Business process automation

The use of software to automate repeatable, multistep business transactions.

Robotic process automation

The use of software robots to perform repetitive tasks previously done by humans.

Industrial automation

The reduction of human labour in manufacturing processes as part of factory automation efforts, usually to the point where human workers simply provide oversight at a control panel or other human-machine interface (HMI).

Artificial intelligence

Rules-based software that performs tasks typically accomplished with human intervention.

Machine learning

Adaptive algorithms that use predictive models to perform tasks without explicit instructions—automatically modifying algorithms with each completed task.

Examples of Adaptive algorithms: **Google Search Algorithm, Predictive Text in Smartphones, Fraud Detection Systems in Banking**

Deep learning

Multiple adaptive algorithms, automation software, and programs that perform a fixed repetitive task—such as extracting small details from raw images.

Why automate?

Automation helps businesses on their path to digital transformation. Organizations today are dealing with major **disruption**—think **Airbnb**, **Amazon**, etc. They're challenged with supporting their employees and partners, reaching new customers, and providing new innovative products and services faster.

Performance measurement through KPIs and Metrics :

Measuring the success of any organization`s DevOps practices are largely dependent on team ability to track and quantify proper **Key Performance Indicator** (KPIs) and other metrics that help evaluate success and identify areas of improvement.

A DevOps transformation often represents a significant investment of time, money, and resources to an organization, and can necessitate an **overhaul** (completely change) of everything from communication to training to tools. Consequently, the ability to assess performance **benchmarks** in a clear and accurate way allows to define objectives, improve efficiency, and track successes to ensure peak productivity.

The intention of a DevOps initiative is to improve collaboration and communication between development and operations teams in a way that facilitates accelerated software delivery while reducing **outages**, **downtime**, and issues that negatively impact the end user experience.

Choosing which key metrics to monitor is **contingent** (depending on) on the specific challenges and needs of any company. DevOps KPIs should provide a comprehensive view that details the impact and business value of DevOps success. Choosing the appropriate performance metrics to track can help guide future production and technology-related decisions while justifying the implementation of existing DevOps efforts.

Deployment frequency

Delivering updates, new features, and software improvements with greater efficiency and accuracy is crucial to building and maintaining a competitive advantage. The ability to improve deployment frequency leads to greater agility and faster **adherence** to changing users' need.

Measuring deployment frequency daily or weekly can provide better insight into which changes were the most beneficial or what areas are still in need of improvement. A sudden decrease in frequency may be a sign that workflow is imbalanced or encumbered by other projects or staffing issues. Deployment frequency metrics that indicate steadiness or small but consistent increases are ideal for sustainable growth and development.

Change failure rate

When it comes to a subject as complex as DevOps, there is no single metric that exists as the sole indicator of success, and deployment frequency is the perfect example. Although increasing frequency seems like one of the ultimate goals of a DevOps transition for greater agility, it must be assessed in conjunction with failure rate. If the more frequent changes being deployed fail too often, the end result could be a loss of revenue and customer satisfaction.

Mean Time to Recovery (MTTR)

MTTR is a key performance indicator that **gauges** any company's efficiency in resolving issues. The ability to evaluate the business impact and customer experience **repercussion** (unintended consequence) provides insight needed to fully understand and prioritize problems. **MTTR** measures average recovery time from failure to resolution, and provides answers about whether customers lost access, experienced errors, or abandoned the application. Improving MTTR reduces impact of said problems, costs and protects customer satisfaction.

Lead time

Measuring the **average time**, it takes for a concept to go from an idea to implementation is an effective metric for evaluating workflow and productivity. Reduced **lead times** indicate that your **DevOps** team is **adaptive**, **productive**, and **capable of addressing feedback promptly**.

The agile methodologies associated with DevOps should enable a short **turnaround time** for system changes, which allows your company to meet customer needs and capitalize on **emerging trends**.

Change volume

The philosophy of **DevOps** supports making changes often, but the metrics for deployment frequency can be misleading if you're not also measuring the volume of change **between deployments**. The end goal shouldn't be a constant stream of minor but insubstantial changes; instead, focus on impactful updates that provide a better experience with less disruption. Tracking the amount of change with each deployment allows for a more accurate representation of progress.

Defect escape rate

Regardless of how experienced your DevOps team is, errors happen -- particularly as changes are being made. Software development requires innovation and defects should be expected and planned for as part of the process.

The defect escape rate is a metric that assesses the collective quality of software releases by evaluating how often errors are discovered and rectified in the pre-production process versus during production.

Customer tickets:

The motivating factor of innovation often involves improving customer satisfaction, and for good reason -- a seamless user experience is good customer service, and frequently translates to an increase in sales. As a result, customer tickets are an effective quantifier of the success of your DevOps transformation. Ideally, customers shouldn’t be serving as quality control by discovering errors and issues, so a reduction in customer tickets is a strong indicator of quality application performance.

Agile and DevOps:

Both DevOps and agile are modern software development practices aimed at providing a framework to produce a part of a product, a launch, or a release.

Understanding the difference

Although both DevOps and agile result in the development of software, they have different approaches, involve different groups and departments, and structure production differently.

The most important thing to know about DevOps versus agile is that they are not mutually exclusive. **DevOps** is a culture, fostering collaboration amongst all participants involved in the development and maintenance of software. **Agile** can be described as a development methodology designed to maintain productivity and drive releases with the common reality of changing needs. Although DevOps and agile are different, when used together these two methodologies lead to greater efficiency and more reliable results.

DevOps definition	**Agile definition**
DevOps is a software development practice that brings people, process, and technology together to deliver continuous value. The approach is divided into planning and tracking, development, build and test, delivery, and monitoring and operations. DevOps is unique in that development, IT operations, quality engineering, and security teams work together to create efficiency across all tasks involved in launching a new product, release, or update.	Agile development is a delivery approach that relates to lean manufacturing. The fundamentals of agile canter around creating a working prototype or build amidst the realities of changing needs and requirements. Bridging the gap between the development team and the end user, adaptability is a core attribute of agile, giving precedence to the needs of users and stakeholders over rigid plans.
DevOps philosophy and focus	**Agile philosophy and focus**
Rooted in stability, consistency, and planning, the DevOps culture seeks to identify new ways to improve and streamline processes. As a result, DevOps focuses on maximizing efficiency, identifying programmable processes, and increasing automation.	The fail-fast mindset of agile canters around adaptability and keeping pace with customer needs and expectations. Features are described as user stories, placing the focus on the individual user, what he or she needs, and why.
DevOps scope	**Agile scope**
DevOps represents the intersections of development, operations, and quality assurance. Cross-disciplinary teams unite and collaborate in the development and delivery of software.	Agile development is specific to the development team, its productivity, and its progress toward completing the project at hand. Development is completed in incremental sprints, and software delivery, deployment, or ongoing maintenance of each release is managed by different teams.
DevOps manifestations	**Agile manifestations**
✔ Continuous integration	✔ Scrum

✔ Continuous delivery ✔ Continuous deployment	✔ Kanban ✔ Lean development ✔ DSDM ✔ Extreme programming ✔ Crystal ✔ Feature-driven development

How DevOps and agile work together

Both DevOps and agile offer a structure and framework that can speed software delivery. You do not need to choose between DevOps or agile—instead, you can make use of both methodologies. Agile is strong on methods to organize work, such as through Scrum or Kanban, and DevOps drives a broader culture of getting software delivered faster and more reliably.

Instead of a decision of DevOps versus agile, the question—actually—is how to practice both. When considering how to build a development practice with the best of DevOps and agile, here are some examples of key benefits and features that can help you create a highly optimized development environment.

Top features from DevOps:

Broader scope, wider reach

DevOps addresses all stages of software development and delivery, seeking to make releases faster and more reliable.

Inter-department collaboration

A culture of reducing friction and fostering cross-functional teamwork can produce an improved work environment and more effective teams.

Efficiency from automation

The DevOps practice seeks to find opportunities to create programmable processes and automate workflows where possible, driving efficiency.

Top features from agile:

Workflow productivity tools

Kanban, Scrum, and other familiar planning tools from agile help track work, helping organize requirements, tasks, and progress.

Incremental progress

Using sprints or other time-boxed production approaches help create a consistent development cadence.

Needs of the customer
Agile's fail-fast, fail-early mentality helps provide a consistent feedback loop, bringing customer expectations to the forefront.

Tools for DevOps and agile

As you build your approach and software delivery practice, you'll want to find a set of tools to best match your workflow. Azure DevOps provides everything you need to plan smarter, collaborate better, and ship faster with a set of modern dev services.

Azure DevOps includes Azure Boards, a set of tools that help you plan, track, and discuss work across your teams. Scrum-ready and Kanban-capable, Azure Boards make it easy to bring agile software development into your DevOps approach.

Azure DevOps lets you customize your experience to fit your workflows—build, test, and deploy with continuous integration and continuous delivery, use proven agile tools to plan and track work, and test and ship with confidence. With modular, mix-and-match tools, this comprehensive suite is flexible for development on any platform.

What is Agile Infrastructure?

Traditional 'waterfall' infrastructure does not allow changes till the very end and is not the best solution for complex and **volatile projects.**
In contrast, an Agile infrastructure is one that supports incremental upgrades, quick deployment and **provisioning**, and allows for feedback and improvements at every stage.

Agile infrastructure is founded on **Agile values** and **principles** and applies them to the hardware and software components of the **organizational IT networks**.

With this infrastructure at the core, teams are easily able to adapt to new technology and collaborate across geographies. They work in small increments using **lightweight deployment** and roll out applications in quick successive releases.

Why the Need for Agile Infrastructure

- ✔ **Rapid Deployment:**
  In a traditional project, processes were linear, and it was only when the project was nearing completion that the product would be released. In direct contrast to this way of working, Agile works on the premise of quick, iterative releases, each of which represents a completed feature or set of features in their entirety. This results in faster deployment and continuous delivery of value and sets in motion an iterative cycle of feedback and improvements.

✔ **More Flexibility:**
Traditional infrastructure is rigid and conservative and does not easily allow for change. However, with Agile, teams can factor in amendments as and when required, course correct at any stage over the course of the project journey and deliver releases that are in sync with customer expectations and the changing needs of the market.

✔ **Continuous Feedback:**
Traditional project methods would typically solicit feedback only after the project was completed. This could result in setting back the final release, and it would often take months to redo the project in accordance with the changes requested. The feedback cycle in Agile Infrastructure allows for stakeholder and end user inputs at every stage. This helps to mould the product according to what the customers need over the course of the project, rather than stick to what was defined during the initial stages.

✔ **Continuous Improvement**:
As Agile believes in rapid deployments, the end users can review and use the product in phases. After each incremental release they give their feedback and the product is improved accordingly, getting fine-tuned at each stage so that it completely meets all the expectations of the customers.

What Is Velocity in DevOps?

Velocity, in the agile methodology, is defined as the **rate of progress in software development from one sprint to the next**. That is, it's the amount of work your team manages to get done within the current sprint, before moving on to the next sprint.

The steps involved in Velocity-based Sprint Planning are as follows:

- Calculate the team's average velocity (from last 3 Sprints)
- Select the items from the product backlog equal to the average velocity
- Verify whether the tasks associated with the selected user stories are appropriate for the particular sprint
- Estimate the tasks and check if the total work is consistent with past sprints.

And what's the point of measuring velocity?

As you continue to keep track of your team's velocity from previous sprints, it'll help you establish a pattern that leads to better budget estimates and **deadline forecasts**. You'll have a better understanding of the time and effort required to complete a new project, which will empower you to make better decisions as a **team lead**.

Azure DevOps comes with built-in functionality that enables project managers and team leads to track the **velocity** of their entire team. You can access a report on your team's velocity both as an **in-context chart** in your product backlog, or as a **universal**

widget on the **team dashboard**. Each report has its own advantages and offers different means to customize the way you estimate your team's velocity.

Moreover, we can:

- Estimate how much amount can be delivered by a particular date
- Estimate a date for a committed amount of work to be delivered
- Understand our goals while fixing the amount of work we will commit for a sprint.

Who all participate in the velocity-based sprint planning?

- A velocity-driven sprint planning meeting involves the **Scrum Master**, **Product Owner**, and all the **development team members**. The **Product Owner** presents the highest-priority product backlog items in the meeting and introduces those **top-priority** items to the team.

- The development team matches their **forecasts** with actual deliverables, estimate accordingly from current sprint to release. And the team's velocity is the most appropriate measure used during forecasting.

How to calculate Sprint Velocity?

Let's assume that the team is doing a one-week sprint to calculate velocity.

Calculating the velocity of sprint 1

- Assume that the team has committed to **5 user stories**
- And each user story= **8 story points**
- Then the total story points in **sprint 1= 40 story points**

Assume that the team has completed **3 user stories** out of 5 by the end of sprint 1, then

- Total user stories completed= **3**
- Total **story points completed= 24** (total no. completed user stories x story points=3 x 8)

Calculating the velocity of sprint 2

- Assume that the team has committed to 7 **user stories**
- And each user story= 8 **story points**

- Then the total story points in sprint 2= **56 story points**

Assume that the team has completed **4 user stories** out of 7 by the end of sprint 2, then

- Total user stories completed= 4
- **Total story points completed= 32** (total no. completed user stories x story points= 4 x 8)

Calculating the velocity of sprint 3

- Assume that the team has committed to **9 user stories**
- And each user story= **8 story points**
- Then the total story points in sprint 2= **72 story points**

Assume that the team has completed 5 user stories out of 9 by the end of sprint 3, then

- Total user stories completed= 5
- **Total story points completed= 40** (total no. completed user stories x story points= 5 x 8)

This is mainly because a new team takes time to get the flow. This also may happen because of some changes happening within a team. The report also demonstrated that in most cases, velocity is likely to stabilize after completing at least three iterations. So, it is good to analyze the completed story points after completing a minimum of 3-5 sprints.

Calculating average sprint velocity

We now know our previous sprint velocities which are given below:

- Sprint 1: 24
- Sprint 2: 32
- Sprint 3: 40

This is our average velocity of past three sprints, which serves as a reference in understanding how the team is completing the user stories in a sprint. This will give you more clarity and help in planning your future sprints perfectly. When you are planning a sprint, velocity will give you the reference as to how many user stories can be done in a sprint.

What is Lean Startup?

Lean Startup is a methodology designed to validate a business hypothesis through short and rapid release cycles of product features, business models, and strategies.

The concept is designed to reduce market risk by validating learning through the release of a Minimum Viable Product (MVP). That is, designing a low-barrier to entry first version of a product, so that it can start driving value sooner, and then iterating on it consistently to add more improvements and functionality.